

MÁSTER UNIVERSITARIO EN TECNOLOGÍAS WEB,
COMPUTACIÓN EN LA NUBE Y
APLICACIONES MÓVILES



VNIVERSITAT
DE VALÈNCIA

MONOGRAFICO

SEMINARIO 2: KAFKA

AUTORES: GUILLERMO BLÁZQUEZ ORTEGA

TUTORA: JUAN GUTIERREZ AGUADO

10-05-2026

Índice general

.1.	Resumen del Seminario	4
.1.1.	Conceptos y Arquitecturas Basicas	4
.1.2.	Kafka	8
.2.	Conclusiones	10

.1. Resumen del Seminario

El siguiente resumen se refiere al bloque 2 de la asignatura de Seminarios. En esta ocasión, hemos tenido la oportunidad de recibir a Pepe Fabra Valverde, empleado de la empresa Capgemini en el puesto de Solution Delivery Office, cuya función principal es resolver dudas y explicar en detalle el funcionamiento de diversas herramientas a otros compañeros de la misma empresa.

Para esta sesión, Pepe nos introdujo a Kafka, una herramienta de gestión de eventos y delivery de mensajes muy potente y muy arraigada al contexto empresarial actual. Dado que Kafka es una herramienta muy completa, de la cual se requiere cierto contexto previo para entender mejor su funcionamiento y su valor, Pepe decidió dividir la sesión en dos partes. Una primera parte en la que se nos introdujo a diferentes conceptos y arquitecturas básicas para la gestión de eventos y el acceso a la capa de persistencia, así como las problemáticas que han motivado la aparición de herramientas como Kafka o RabbitMQ. Y una segunda parte en la que nos adentramos de forma profunda en Kafka, su funcionamiento interno, el conjunto de elementos que la componen y su filosofía de funcionamiento.

.1.1. Conceptos y Arquitecturas Basicas

Para la primera parte del seminario, Pepe nos mostro diferentes conceptos basicos relativos al acceso a la capa de persistencia, el valor del dato y los conceptos relativos a la mensajería de eventos.

Teorema CAP

El primero de los conceptos que nos explicó fue el teorema *CAP*, el cual fue planteado por Eric Brewer, que es uno de los teoremas fundamentales de la computación. Este plantea que un sistema puede ser Consistente, Disponible o con Tolerancia Parcial, pero no los tres simultáneamente. Popularmente se representa con un triángulo y dependiendo de hacia qué vértice te muevas, tu sistema ganará uno o dos de las tres virtudes y perderá la tercera sobrante.

4.1

Este teorema se utiliza principalmente en la capa de persistencia, en el contexto de la computación ya que describe a la perfección lo que sucede al tener bases de datos distribuidas y replicadas las cuales almacenan información cuyo nivel de veracidad puede variar dependiendo de para qué se utilice dicha información.

Event Driver Architecture

Una vez explicado el teorema *CAP*, Pepe paso a explicar la arquitectura Event-Driven, la cual se emplea en el contexto de los sistemas con desacoplamiento. Estos sistemas permiten un mejor escalado del sistema, und desacoplamiento entre diferentes funciones y mantener una asincronía en el sistema sin que se produzcan pérdidas de información.

Es por ello que surge la arquitectura Event-Driven como una forma de establecer normas, reglas y unidades, con propiedades que definan cómo y qué sucede cuando dos sistemas desacoplados interactúan, así como los efectos secundarios resultantes de dicha

comunicación. Esto logra añadir una capa de control en el sistema y conservar la asincronía del mismo, siempre que el sistema requiera de dicha asincronía.

Uno de los conceptos clave de esta arquitectura es la aparición de los roles

Publisher (*Core del Programa*) y Suscriptor (*Servicio con el que se comunica el core*). También es necesario mencionar los conceptos que sustentan las interacciones entre Publicador y Suscriptor. A continuación, podemos ver los conceptos que sustentan las interacciones entre el publicador y el suscriptor.

- **Evento:** Describe una acción que ha sucedido y que es inmutable.
- **Stream:** Secuencia ordenada de *Eventos* dentro de una instancia.
- **Event Journal:** Almacena el conjunto de streams de forma persistente, sin poder modificarlo ni borrarlo.

Entre las principales ventajas que la arquitectura Event-Driven nos ofrece, Pepe destacó principalmente:

- **Comunicación y Mensajería:** Garantiza la entrega de mensajes entre componentes, y apartes del sistema incluso aunque se produzcan imprevistos o errores.
- **Monitorización y Logs:** Permite tener una mayor visión de conjunto de lo que ha sucedido y está sucediendo en el sistema a través de la recolección de logs.
- **Tratamiento de datos y réplicas:** Permite monitorizar la inserción de datos y mantener el sistema sincronizado, manteniendo la integridad de estos en tiempo real de generación.

Una vez Pepe terminó de explicar las bases de la arquitectura Event-Driven y sus principales ventajas, nos explicó las arquitecturas que se basan en Event-Driven, las cuales están altamente relacionadas con Kafka. Los siguientes subapartados desglosan las características más importantes que durante el seminario Pepe destacó.

Arquitectura de Notificaciones

La arquitectura de Notificaciones es simple pero esencial, ya que esta define como los componentes deben actuar una vez genera o está involucrado en un Evento. De esta forma, se establece que por lo general la comunicación no debe ser bloqueante entre componentes, y que una vez enviada o derivada una tarea a otro componente el *publisher* debe entenderse hasta recibir la respuesta del *subscriber* en caso de que se espere una respuesta de este.

La principal ventaja de esta arquitectura es que permite mantener los componentes desacoplados, aunque conectados entre sí, lo que aporta una mayor flexibilidad y escalabilidad al sistema.

Pepe destacó que, si esta arquitectura se implementa correctamente, ante una caída repentina del sistema debería ser posible restaurar el estado previo al fallo. Sin embargo, aunque esto sea viable desde un punto de vista teórico, en la práctica resulta complejo de conseguir, ya que los eventos carecen de semántica propia. Por ello, el patrón debe aplicarse teniendo en cuenta esta limitación.

Otro inconveniente destacable es que puede generarse cierto acoplamiento entre componentes, puesto que, para conocer el estado del sistema o las acciones realizadas tras un evento, es necesario consultar al componente ejecutor. Esto puede afectar, en cierta medida, a la flexibilidad característica de la arquitectura *Event-Driven*.

CQRS

El patrón *Command Query Responsibility Segregation* (CQRS) se aplica principalmente a nivel de persistencia de datos y establece que las operaciones de lectura y escritura deben realizarse de forma separada. Para ello, se utilizan tablas o modelos diferenciados, donde cada uno está especializado en un único tipo de operación.

En este contexto, se definen dos conceptos principales: *Command* y *Query*. Por un lado, *Command* representa todas las acciones relacionadas con la escritura y modificación de datos, las cuales generan cambios en el estado del sistema. Por otro lado, *Query* engloba las operaciones de lectura, cuya finalidad es únicamente consultar información sin alterar los datos almacenados.

La separación entre lectura y escritura implica que el sistema debe mantener estructuras de datos independientes, correctamente dimensionadas y replicadas en función del volumen de operaciones de cada tipo. Sin embargo, este enfoque también introduce ciertas problemáticas relacionadas con la consistencia de los datos, ya que mantener sincronizados ambos modelos puede resultar complejo. Como consecuencia, suelen aparecer mecanismos adicionales para tratar de garantizar dicha consistencia, lo que en muchos casos termina generando cierto grado de acoplamiento entre componentes.

Pepe comentó que este patrón resulta especialmente útil e interesante en sistemas que dependen de APIs o manejan volúmenes considerables de datos, ya que permite optimizar de forma independiente las operaciones de lectura y escritura. No obstante, también destacó que su adopción condiciona de forma importante las arquitecturas y modelos de trabajo a implementar en el sistema.

Event-Carried State Transfer

El patrón *Event-Carried State Transfer* (ECST) es una arquitectura de sincronización de datos basada en eventos que permite a los consumidores mantener una copia local y actualizada del estado de una entidad sin necesidad de consultar continuamente al servicio productor de forma síncrona. De esta forma, cada servicio puede trabajar con sus propios datos replicados, reduciendo la dependencia directa entre componentes.

A diferencia de una arquitectura de notificaciones tradicional, en ECST el evento no solo informa de que algo ha ocurrido, sino que también transporta la información necesaria para actualizar el estado del consumidor. Este estado puede enviarse de dos formas: mediante el estado completo de la entidad o únicamente a través de los atributos que han cambiado respecto a la versión anterior, conocidos como *deltas*.

Habitualmente, este patrón se combina con arquitecturas *CQRS* y con mecanismos como *Event Journal* o *Outbox*, los cuales actúan como fuente de verdad y permiten reconstruir el estado de forma autónoma a partir del histórico de eventos.

Entre sus principales ventajas destaca el bajo acoplamiento entre servicios, ya que el consumidor no depende directamente de la disponibilidad del productor para consultar

datos. Además, mejora la escalabilidad y reduce la latencia de lectura al trabajar sobre copias locales o vistas materializadas. También aporta una mayor tolerancia a fallos y particiones de red, permitiendo que el sistema continúe funcionando incluso si algunos servicios dejan de estar disponibles temporalmente.

Sin embargo, ECST también introduce ciertas complejidades. La más importante es la *consistencia eventual*, ya que puede existir un desfase temporal entre el cambio original y su propagación al resto de consumidores. A esto se suma la necesidad de gestionar versiones de eventos, mantener múltiples almacenes de datos sincronizados y controlar problemas derivados de la doble escritura (*Dual Write*), donde el productor debe actualizar su base de datos y publicar el evento de forma atómica.

La aplicación de ECST puede adaptarse según el dominio y las necesidades del sistema. Es habitual utilizarlo para sincronizar bases de datos, mantener réplicas de lectura, compartir información entre microservicios o construir vistas materializadas optimizadas para consultas complejas. Además, combinado con *Event Sourcing*, permite reconstruir el estado histórico de una entidad en cualquier momento.

En conclusión, ECST es un patrón especialmente útil en arquitecturas distribuidas y de microservicios, ya que permite mejorar el desacoplamiento, la escalabilidad y el rendimiento del sistema. No obstante, su adopción implica aceptar modelos de consistencia eventual y asumir una mayor complejidad en la gestión de eventos y sincronización de datos.

Event Sourcing

Para finalizar, Pepe nos explico la arquitectura *Event-Driven* (EDA) la cual representa la consolidación de los patrones de arquitectura basados en eventos. Mientras que técnicas como la *Arquitectura de Notificaciones*, *Event-Carried State Transfer (ECST)* o *Event Sourcing* abordan necesidades específicas (comunicación, sincronización o persistencia), la EDA se define como el **paradigma unificador** que abarca todas estas variantes bajo un mismo modelo de diseño.

En este contexto, la arquitectura dirigida por eventos (EDA) se posiciona como una arquitectura distribuida madura, capaz de unificar distintos patrones bajo un modelo de comunicación basado en eventos y *Publish/Subscribe* (Pub/Sub). Su objetivo es crear sistemas desacoplados, escalables y resilientes, donde los componentes operen de forma asíncrona e independiente. Para ello, el evento actúa como la entidad central de la arquitectura, permitiendo comunicar cambios de estado sin dependencias directas entre servicios. Además, la persistencia de eventos en *streams* facilita la reconstrucción de estados y mejora la tolerancia a fallos del sistema.

La adopción de EDA como arquitectura ofrece beneficios transformadores para los sistemas distribuidos. El bajo acoplamiento permite que los sistemas evolucionen, se actualicen o se reemplacen sin afectar a otras partes del ecosistema, siempre que mantengan el formato de los eventos. Esto se combina con una alta escalabilidad, que facilita escalar horizontalmente tanto a productores como a consumidores de forma independiente según la carga de eventos.

Además, la arquitectura garantiza tolerancia a particiones y fallos; gracias a su naturaleza asíncrona y al uso de brokers de mensajes como Kafka, el sistema sigue operando incluso ante fallos de red o nodos individuales. La reactividad en tiempo real habilita el procesamiento de flujos de datos continuos, permitiendo respuestas inmediatas a cambios

de estado en el negocio. Por último, ofrece una excelente observabilidad y trazabilidad, ya que el Event Journal proporciona un historial completo de cambios que facilita auditorías, depuración y análisis forense del sistema.

La arquitectura dirigida por eventos (EDA) no es una solución mágica, sino un enfoque que introduce una elevada complejidad técnica y operativa. Su principal característica, la consistencia eventual, implica que el sistema no mantiene un estado global instantáneo, ya que la información se propaga de forma asíncrona entre componentes.

Esto dificulta tareas como la depuración, el trazado de eventos y la gestión de versiones, además de requerir infraestructuras robustas y mecanismos capaces de garantizar la entrega y el orden de los mensajes. Por ello, la adopción de EDA no solo supone un coste tecnológico, sino también un cambio importante en la forma de diseñar y desarrollar software.

La arquitectura dirigida por eventos (EDA) actúa como una arquitectura base capaz de integrar distintos patrones y modelos orientados a resolver problemas complejos de software. Esta permite implementar arquitecturas de notificaciones no bloqueantes, sincronizar datos entre dominios mediante Event-Carried State Transfer (ECST), registrar cambios de estado a través de Event Sourcing y mantener sincronizados los modelos de lectura y escritura en arquitecturas CQRS. De esta forma, la EDA se convierte en el mecanismo principal de comunicación y propagación de eventos dentro del sistema.

La *Event-Driven Architecture* se presenta como la arquitectura definitiva para sistemas modernos que demandan escalabilidad, resiliencia y reactividad. Al adoptar EDA, las organizaciones aceptan la *consistencia eventual* a cambio de una libertad de diseño sin precedentes y una capacidad de adaptación ante cambios de negocio rápidos. No es simplemente una tecnología de mensajería, sino un cambio de paradigma en la forma en que los sistemas distribuidos interactúan, evolucionan y se mantienen.

.1.2. Kafka

Una vez finalizado la explicación de conceptos básicos, Pepe continuó explicándonos Kafka detalladamente. Esta herramienta pertenece a Apache y es una **plataforma de mensajería basada en streaming** que funciona esencialmente como un almacén de eventos o una cola distribuida. Su propósito principal es permitir la comunicación asíncrona mediante un paradigma *publicador-suscriptor*, donde los eventos son la unidad fundamental de intercambio de información.

Características

Kafka cumple con los requisitos de una arquitectura orientada a eventos y ofrece ventajas interesantes al sistema. Entre las más interesantes destacan; Un bajo nivel de acoplamiento entre componentes del sistema; Un alto rendimiento y disponibilidad, soportando un gran volumen de eventos simultáneos; Y tolerancia a particionamiento, lo que supone una gran resiliencia del sistema y escalabilidad indirectamente.

Debido a que Kafka emplea colas para la distribución de mensajes de forma secuencial, es capaz de manejar millones de eventos simultáneamente. Es por ello que se emplea en las principales arquitecturas del sistema:

1. Sistemas de aviso y comunicación en tiempo real.

2. Agregación de métricas y logs.
3. Sincronización de sistemas y datos.

Kafka también ofrece otras múltiples ventajas que han llevado a su rápida implementación en el tejido empresarial. En primer lugar, se trata de una tecnología *Open Source*, lo que permite acceder públicamente a su código y facilita su integración y evolución. Además, dispone de distribuciones gratuitas como Apache Kafka, junto con alternativas comerciales como Confluent, que añaden funcionalidades adicionales. Otro aspecto fundamental es su capacidad para gestionar volúmenes masivos de datos, llegando a procesar transferencias de varios petabytes al día. Finalmente, Kafka está especialmente orientado a entornos *Big Data*, pudiendo ejecutarse de forma distribuida sobre infraestructuras como Hadoop compuestas por miles de nodos ligeros.

Pepe nos comentó una de las implementaciones más conocidas de Kafka, la cual se trataba de Netflix que en 2016 anunció un volumen estimado de 550 mil millones de eventos diarios y picos de 8.5 millones de eventos por segundo, escalando posteriormente a 1 trillón de eventos diarios.

Componentes de Kafka

La arquitectura de Kafka se compone de diversos elementos diseñados para satisfacer distintos requisitos del sistema y adaptarse a diferentes contextos de implementación. Cada uno de estos componentes ofrece ventajas y desventajas específicas, las cuales deben evaluarse según el entorno arquitectónico en el que se despliegan.

En cuanto a los componentes básicos, la arquitectura se compone de varios elementos fundamentales. El *Producer* es el cliente encargado de generar y enviar eventos al sistema, mientras que el *Consumer* es el componente responsable de consumir o leer dichos eventos. El intercambio de información se realiza mediante los *Events*, que representan los mensajes transmitidos y están compuestos por una *Key* y un *Body*. Estos eventos se almacenan dentro de un *Topic*, el cual actúa como una secuencia ordenada de mensajes persistidos en disco siguiendo un modelo FIFO. Finalmente, cada consumidor mantiene un *Offset*, que identifica el índice del último evento leído dentro de un *topic* determinado.

Por otra parte, los componentes de clúster de Kafka están compuestos por distintos elementos encargados de garantizar la distribución y persistencia de los eventos. El componente principal es el *Broker*, un nodo virtual encargado de almacenar y gestionar los eventos dentro del clúster, normalmente desplegado en configuraciones activo-activo para mejorar la disponibilidad. Cada *Topic* se divide en varias *Partitions*, las cuales son gestionadas de forma exclusiva por un broker, permitiendo así aumentar la escalabilidad y el rendimiento del sistema. Para decidir a qué partición debe enviarse cada evento, Kafka utiliza un *Partitioner*, un algoritmo que distribuye los mensajes normalmente en función de una clave. Finalmente, *ZooKeeper* actúa como el servicio de coordinación del clúster, encargándose de gestionar el estado de los brokers, los *topics* y las particiones.

Por último, los componentes de Replicación y Tolerancia garantizan la disponibilidad y la tolerancia a fallos, el sistema emplea distintos mecanismos de replicación y confirmación de mensajes. En primer lugar, las *replicas* actúan como copias de seguridad de una partición, donde cada una es gestionada por un *broker* diferente. Dentro de estas réplicas, únicamente un broker funciona como *leader* activo, mientras que el resto permanecen como respaldo ante posibles fallos.

Por otra parte, los *Consumer Groups* permiten agrupar múltiples consumidores bajo una misma unidad lógica, compartiendo el mismo *offset*. Esto facilita el procesamiento paralelo de eventos y mejora la escalabilidad del sistema.

Finalmente, el parámetro *ACK* (*Acknowledgement*) define el número de réplicas que deben confirmar la escritura de un mensaje antes de que el *producer* reciba una respuesta satisfactoria. Un valor *ACK* = 0 prioriza el rendimiento, aunque sin garantías de persistencia. Con *ACK* = 1, únicamente el *leader* confirma la operación, ofreciendo un equilibrio entre rendimiento y fiabilidad. Por último, *ACK* = ALL exige la confirmación de todas las réplicas, proporcionando la máxima consistencia y tolerancia a fallos, aunque sacrificando parte del rendimiento.

.2. Conclusiones

Kafka es una herramienta excepcional, potente y flexible, la cual está enfocada para el manejo masivo de eventos en sistemas distribuidos. No obstante, no es una herramienta perfecta, ya que Pepe nos comentó que todos los aspectos de la configuración deben ser definidos en archivos de texto y que es necesario definir muchos atributos que luego no nos son necesarios ni útiles en tiempo de ejecución.

Esto supone que para la implementación de Kafka se debe invertir tiempo en el diseño y la implementación de los componentes, así como tiempo de configuración y testeo para evitar errores en producción.

Es por este motivo que Pepe nos remarcó que Kafka no es la solución ideal para todos los sistemas, que es más adecuado para sistemas complejos con muchos eventos y muchos componentes, y que para sistemas menos complejos es suficiente con emplear RabbitMQ o cualquier otro gestor de mensajes más sencillos de implementar y de mantener.

El seminario de Pepe fue muy interesante y personalmente logró resolver muchas dudas acerca de Kafka. Además, nos dio una batería de documentación, papers y libros para comprender mejor los paradigmas de las arquitecturas de mensajería.

Índice de comentarios

4.1 Tolerancia a Particiones